RFC: Store Versioned Simulation Content in a Validated Hybrid Model

Field	Value
Author	Kenneth Robertson
Date	2026-10-20
Status	Draft
Decision Date	To be completed when decided
Deciders	OSU Capstone team; OHSU project partner confirms requirements and confidentiality impacts
Superseded By	None

Problem and Context

The OHSU Simulation Database must preserve detailed cases while making them searchable by campus, course, clinical topic, learner level, modality, competency, and learning outcome. A case includes patient information, equipment, participant roles, scenario progression, cues, expected actions, debriefing, assessments, and supporting materials. These sections may change as OHSU's template is refined, while approved cases still need version history, peer review, curricular mapping, and predictable search. The team must decide which parts belong in relational tables and which should remain a structured document.

Pull request #3 introduces the first database schema. It normalizes users, campuses, courses, outcomes, case versions, tags, usage, and reviews, but stores the template body in `case_version.content JSONB` with no `NOT NULL` requirement, schema identifier, or structural validation [1]. The pull request is still open, so this RFC can change it. After faculty cases are imported, changing representation would require migrating every historical version and handling records that cannot be mapped automatically. That makes this a hard-to-reverse decision.

If the team does nothing, each feature may make different assumptions about keys inside `content`. Versions could use different names or types for the same concept and still be valid JSON. Relational and JSON values could disagree. The cost arrives when the team builds editing, imports, exports, review comparisons, or annual maintenance, when correction could threaten case history.

Options

Option A: Validated Hybrid Relational/JSONB Model

Keep stable identity, workflow, permissions, searchable metadata, relationships, and audit information in relational tables. Keep the evolving, version-specific simulation template body in `JSONB`. Add a required `content_schema_version` to every case version, require `content` to be non-null, define a versioned JSON Schema in the repository, and validate every write at the trusted service boundary. The application must treat relational metadata as authoritative and must not duplicate those fields in JSON unless an explicitly documented snapshot is required.

This option fits the current schema direction and the project's evolving template. It allows a new optional template section without immediately adding several tables and joins. JSON Schema can state required properties, types, enumerations, array item shapes, and whether unknown properties are allowed [5].

PostgreSQL also provides native JSONB operators and GIN indexes if a future, measured query must inspect document content [4].

The costs are two validation systems: relational constraints and JSON Schema. The team must maintain schema versions and migrations or compatibility readers. Cross-document constraints and arbitrary reporting from nested JSON are harder. The boundary between relational data and document content must be documented and tested.

Option B: Fully Normalize the Simulation Template

Represent each significant section and repeated item as relational data: for example, case objectives, patient history, medications, equipment, participant roles, scenario states, cues, actions, responses, debriefing prompts, and assessments would receive columns or child tables. All case content would be connected to a case version by foreign keys.

The benefit is strong database-level integrity. Required relationships can be expressed with `NOT NULL`, `UNIQUE`, `CHECK`, and foreign-key constraints, and SQL reporting can address every modeled field directly. PostgreSQL documents that foreign keys preserve referential integrity and that primary or unique constraints identify referenced rows [6]. A fully relational model may be best if the OHSU template is stable and the partner needs frequent cross-case reporting on most individual fields.

The repository guidance says the domain vocabulary is not a final schema and architecture details remain unresolved [2]. Normalizing every nested section would encode assumptions about a changing template. A form change could require database, model, API, and import/export migrations. Ordered scenario steps may also need many tables without proven query value. This option has the greatest up-front burden.

Option C: Do Nothing - Merge the Current Unvalidated JSONB Design

Accept pull request #3 without adding a content schema version or structural validation contract. Application code would read and write whatever JSON shape its current form expects. Stable metadata would remain relational as already drafted.

This is credible for a disposable prototype. It has the lowest immediate cost and avoids validation design before the OHSU template is complete.

The cost is deferred. Inconsistent versions could accumulate silently. Imports and UI revisions could not identify document shape reliably. Review diffs could compare different structures, and exports could omit fields without failing. Retrofitting a schema version would require guessing existing formats. That conflicts with preserving versions and converting existing cases.

Analysis

Data integrity and change over time

Full normalization provides the strongest database enforcement only after stable entities are known. That condition is not yet met for the entire template; the guide warns that its domain vocabulary is not a final schema [2]. Premature normalization would make uncertain details look settled.

The do-nothing design gives JSON syntactic validity but no shared meaning. `JSONB` alone does not require a `debriefing` section, constrain a medication entry, or distinguish template versions. JSON Schema Draft 2020-12 supplies structural validation [5]. A separate `content_schema_version` lets code select the correct validator and migration rather than infer a version.

Hybrid storage does not replace relational integrity. Ownership, version numbers, current-version pointers, reviews, course use, outcomes, and search facets should remain relational. The draft mostly follows this pattern [1]. The team must also ensure `sim_case.current_version_id` belongs to that same case; the current foreign key proves only that the version exists.

Search and performance

The primary search filters named by the partner should use normalized columns and join tables. This gives predictable equality and combination filters and avoids depending on arbitrary JSON paths. PostgreSQL notes that foreign keys do not automatically create indexes on referencing columns, so query-specific indexes still require deliberate design [6]. The current schema has some indexes but the team has not yet supplied production-size data, query plans, or latency targets. I therefore cannot claim that any option is faster for the final workload.

PostgreSQL supports GIN indexing over JSONB keys and key/value pairs [4], but the team should not index the entire document in advance. It should measure representative synthetic queries with `EXPLAIN (ANALYZE, BUFFERS)`. A frequently filtered JSON field should be promoted to relational metadata or receive a targeted index.

Full normalization makes every field queryable but adds joins and complex writes. Do nothing has similar potential speed to Option A, but inconsistent keys undermine correctness. Stable query semantics favor Option A.

Maintainability and complexity

Option A concentrates evolution in versioned schemas and compatibility code. Reviewers can compare `v1` and `v2`, and tests can run fixtures through both. Option B distributes changes across SQL, models, APIs, and forms. Option C transfers complexity to every consumer of unvalidated content.

Fields used for authorization, workflow, approval, audit, search, or cross-case reporting cannot live only in JSON. The application must not copy a relational value into JSON and allow both to change independently.

Security, privacy, and confidentiality

Authorization must still be server-side for reads, writes, reviews, exports, and history. Repository rules prohibit real personal records in development and require synthetic test data [2]. Validation should reject unknown properties where practical, limiting unintended collection. Errors must identify a failing path without printing the complete case.

Option B constrains more values but does not remove confidentiality obligations. Option C cannot reliably state what a case may contain. Option A provides a reviewable inventory without a premature table for every field.

Cost and reversibility

Before real imports, all options are reversible. Later, Option C is expensive because rows have no declared format; Option B requires reconstructing many related records. Option A creates a migration seam: each record declares its version, and idempotent upgrades retain the original case version.

The main uncertainty is the final template and actual reporting fields. The repository has no application, tests, CI, production-like dataset, or measured workload. This recommendation relies on documented requirements and platform capabilities, not invented benchmarks.

Recommendation and Verification Plan

I recommend Option A. It balances relational search, workflow integrity, and version history with an evolving template, while changing the draft incrementally.

The implementation should:

1. Keep case identity, ownership, version number, status, review dates, users, campus/course use, tags or controlled vocabularies, outcomes, reviews, and current-version selection relational.

2. Change `case_version.content` to `JSONB NOT NULL` and add `content_schema_version INTEGER NOT NULL` with an explicit supported range or reference.
3. Add a versioned JSON Schema such as `schemas/simulation-case/v1.schema.json`, using a declared Draft 2020-12 dialect.
4. Validate content before every create, import, update, or new-version write. Return field-level errors without logging the full document.
5. Treat an approved version as immutable; edits create a new version rather than changing the stored body in place.
6. Document which fields are relational and prohibit unsynchronized duplicates in JSON.

The team will know the decision worked when these acceptance criteria pass:

- A known-good synthetic case validates and can be stored, read, exported, and validated again without losing data.
- Fixtures missing each required top-level section, using a wrong type, or containing a prohibited unknown property are rejected before persistence.
- Every stored case version has a recognized `content_schema_version`; the application refuses an unknown future version safely rather than guessing.
- A version-1 fixture can be upgraded by an idempotent migration, while the original historical version remains readable and unchanged.
- Search by campus, course, clinical topic, learner level, modality, competency, and outcome uses relational metadata and returns correct results for combined filters.
- A database integration test proves that a case's `current_version_id` cannot reference another case's version.
- Authorization tests prove that an unauthorized user cannot read, edit, review, approve, or export a case or its history.
- Validation and request logs contain no complete patient-chart or free-text case body.
- With a synthetic dataset sized to an agreed estimate, representative combined-filter queries meet a team-approved latency target and their query plans use intended indexes. The target must be set with the partner or team before measuring; this RFC does not invent one.

These checks should run on every pull request once the test and CI framework exists. Before that infrastructure exists, the schema change must receive at least one teammate review, as required by `CONTRIBUTING.md`, and the author must record the exact manual validation performed [3]. The rollback point is before importing non-disposable cases: the team can revert the migration and schema files while only synthetic fixtures exist. After import, rollback requires a tested export/restore or reverse migration and a record count plus validation report before and after.

Constraints

The public repository may contain team-written code, schemas, synthetic fixtures, and public documentation. It must not contain partner-confidential documents, OHSU endpoints, credentials, production exports, or identifiable records [2]. JSON examples must use fictional data. Partner templates may inform requirements within the approved boundary; this RFC must not reproduce confidential text.

The current draft uses PostgreSQL-specific `JSONB`, so accepting this RFC would create a PostgreSQL commitment unless the team abstracts or revisits it. PostgreSQL's permissive license does not impose a reciprocal source-code obligation, but deployment and managed-service costs remain undecided. The

repository currently has no project `LICENSE` file, so this RFC does not authorize redistribution of partner materials or settle ownership of the final deliverable. Any OHSU or OSU IP and retention requirements override convenience.

Delegation and Validation

I used ChatGPT/Codex to inspect the repository and open pull request, identify candidate hard-to-reverse decisions, compare the three options, locate primary technical documentation, and produce the first written draft. The AI was not authorized to merge pull request #3, accept this RFC, select a technology stack for the team, or generate claims about benchmarks that were never run. This boundary was deliberate because the decision affects future migrations and partner data.

The repository currently provides a human review gate in `CONTRIBUTING.md`: one approval from someone other than the author is required, and schema changes require review under `AGENTS.md` [2][3]. However, no CI workflow, test suite, staging environment, or executable rollback migration currently exists in the repository. I am stating that absence rather than claiming intended safeguards as completed ones. The immediate validation available for this draft is comparison against the actual `schema.sql`, the project's repository guidance, and the cited PostgreSQL and JSON Schema documentation.

Before submitting or voting on the RFC, I will personally perform three checks: trace every proposed relational field and JSON responsibility back to a project requirement; open each cited source and confirm it supports the adjacent claim; and walk through one synthetic case from draft to approved revision, including search, review, export, and schema upgrade. I will record corrections from that walkthrough in the RFC. I am also stopping for team and OHSU-partner review before any real-data migration because only they can confirm the final template, reporting needs, confidentiality boundary, and acceptable deployment commitment. If I cannot explain the model, its failure modes, and its rollback path without the AI tool, the draft is not ready for acceptance.

Decision

Pending. After review, record accepted, rejected, or superseded, the decision date, the person who chaired the decision, and a link to any replacing RFC. Until then, pull request #3 should be treated as a proposal rather than proof that the content model has been accepted.

References

1. OSU Capstone OHSU Database, “Create schema.sql,” pull request #3 and proposed `db/schema.sql`, 2026. https://github.com/robekenn/OSU-Capstone-OHSU-Database/pull/3
2. OSU Capstone OHSU Database, “OHSU Simulation Database Agent Guide,” `AGENTS.md`, 2026. https://github.com/robekenn/OSU-Capstone-OHSU-Database/blob/main/AGENTS.md
3. OSU Capstone OHSU Database, “Contributing,” `CONTRIBUTING.md`, 2026. https://github.com/robekenn/OSU-Capstone-OHSU-Database/blob/main/CONTRIBUTING.md
4. PostgreSQL Global Development Group, “JSON Types: jsonb Indexing,” PostgreSQL 18 Documentation. https://www.postgresql.org/docs/current/datatype-json.html#JSON-INDEXING
5. JSON Schema, “JSON Schema Specification, Draft 2020-12.” https://json-schema.org/specification
6. PostgreSQL Global Development Group, “Constraints,” PostgreSQL 18 Documentation. https://www.postgresql.org/docs/current/ddl-constraints.html